

Mastering FastAPI

Building Production APIs from Scratch with SQLAlchemy

A complete, hands-on guide from beginner to advanced — theory and implementation, with full CRUD operations.

How to Read This Book

This book takes you from knowing nothing about FastAPI to building and deploying complete, production-grade APIs. Every chapter pairs **theory** (why something works the way it does) with **implementation** (runnable code you can type and execute).

The running example throughout the book is a **Bookstore API** — you will build it incrementally, chapter by chapter, until it is a fully featured application with authentication, a real database, tests, and a deployment pipeline.

Conventions used in this book

- Code you should run in a terminal is prefixed with `$`.
- Python files are named at the top of each block (e.g., `# main.py`).
- "Try it" callouts invite you to experiment.
- Each chapter ends with a recap and exercises.

What you need

- Python 3.10 or newer (3.11+ recommended).
 - Basic familiarity with Python syntax (functions, classes, dictionaries). If you can read a `for` loop and define a function, you are ready.
 - A terminal and a text editor (VS Code recommended).
-

Table of Contents

Part I — Foundations 1. What Is FastAPI and Why It Exists 2. Setting Up Your Environment 3. Python You Need: Type Hints and Async 4. Your First FastAPI Application 5. Path Parameters, Query Parameters, and Request Bodies

Part II — Building Real APIs 6. Pydantic: Data Validation Done Right 7. SQLAlchemy and Databases 8. Building a Complete CRUD API 9. Relationships Between Models 10. Dependency Injection 11. Error Handling and Validation

Part III — Production-Grade Features 12. Authentication and Authorization with JWT 13. Async Databases and Performance 14. Background Tasks and WebSockets 15. Testing Your API 16. Middleware, CORS, and Configuration 17. Deployment 18. Capstone: The Complete Bookstore API

Appendices - A. HTTP Status Codes Reference - B. Common Pitfalls and How to Avoid Them - C. Further Reading

Part I — Foundations

Chapter 1 — What Is FastAPI and Why It Exists

1.1 The problem FastAPI solves

A web API is a program that receives HTTP requests and returns HTTP responses, usually in JSON. Before FastAPI, Python developers building APIs faced a tradeoff. Frameworks like Flask were simple but gave you almost no help: you validated incoming data by hand, wrote your own documentation, and converted types manually. Frameworks like Django REST Framework were powerful but heavy and verbose.

FastAPI, created by Sebastián Ramírez and first released in 2018, took a different approach. It uses standard Python **type hints** as the single source of truth. From those hints it automatically does three things that used to require enormous manual effort:

1. **Validates and parses** incoming request data.
2. **Serializes** your responses into JSON.
3. **Generates interactive API documentation** automatically.

The result is that you write less code, make fewer mistakes, and get features for free that other frameworks make you build.

1.2 The pillars FastAPI stands on

FastAPI is not built from scratch. It composes two excellent libraries:

- **Starlette** handles the web machinery: routing, requests, responses, WebSockets, middleware. It is the ASGI foundation underneath FastAPI.
- **Pydantic** handles data: it defines the shape of your data using Python classes and validates that incoming data matches.

FastAPI is the glue that connects them and adds the magic of automatic documentation and dependency injection. When you understand that FastAPI = Starlette (web) + Pydantic (data) + glue, the whole framework becomes far less mysterious.

1.3 What "ASGI" means and why it matters

Traditional Python web frameworks used **WSGI** (Web Server Gateway Interface), a standard built around synchronous, one-request-at-a-time handling. It works, but it struggles when a server must handle thousands of simultaneous connections that

spend most of their time waiting — waiting for a database, an external API, or a file.

ASGI (Asynchronous Server Gateway Interface) is the modern successor. It supports `async/await`, allowing a single process to handle many requests concurrently by switching to other work while one request waits. FastAPI is an ASGI framework, which is a large part of why it is fast. We will cover `async` in depth in Chapter 3.

1.4 Why "Fast"

The name refers to two kinds of speed:

- **Runtime speed.** Because it is `async` and built on Starlette and Pydantic (whose core is compiled), FastAPI is among the fastest Python frameworks, comparable to Node.js and Go for many workloads.
- **Development speed.** Type hints, autocompletion, and automatic docs mean you build correct software faster and debug less.

1.5 The automatic documentation

This is the feature that wins people over. Every FastAPI app exposes, with zero extra code, two interactive documentation interfaces:

- **Swagger UI** at `/docs` — lets you read and *try out* every endpoint in the browser.
- **ReDoc** at `/redoc` — a clean, readable reference.

Both are generated from the **OpenAPI** standard (formerly Swagger), a machine-readable description of your API that FastAPI builds automatically from your code.

1.6 Recap

FastAPI is a modern Python web framework that turns type hints into validation, serialization, and documentation. It is built on Starlette (`async web`) and Pydantic (data validation), runs on ASGI, and is fast both at runtime and during development. In the next chapter you will install it and prepare your workspace.

Chapter 2 — Setting Up Your Environment

2.1 Checking your Python version

Open a terminal and run:

```
$ python --version
```

You should see `Python 3.10` or higher. If `python` points to Python 2 on your system, try `python3`. FastAPI uses modern type-hint syntax that works best on 3.10+.

2.2 Why you must use a virtual environment

A virtual environment is an isolated folder containing its own copy of Python packages. Without one, every project shares the same global packages, and upgrading a library for one project can silently break another. Always create a fresh environment per project.

```
$ mkdir bookstore-api  
$ cd bookstore-api  
$ python -m venv venv
```

Activate it:

```
# macOS / Linux  
$ source venv/bin/activate  
  
# Windows (PowerShell)  
$ venv\Scripts\Activate.ps1
```

When active, your prompt shows `(venv)`. Everything you install now lives only in this folder.

2.3 Installing FastAPI

The simplest install brings FastAPI plus the standard optional dependencies (including the Uvicorn server):

```
$ pip install "fastapi[standard]"
```

This single command gives you FastAPI, Uvicorn (the ASGI server that runs your app), and helpful extras. For the database chapters you will also install SQLAlchemy:

```
$ pip install sqlalchemy
```

2.4 Freezing dependencies

Record exactly what you installed so others (and your future self) can reproduce the environment:

```
$ pip freeze > requirements.txt
```

Later, anyone can recreate it with `pip install -r requirements.txt`.

2.5 Understanding the server: Uvicorn

Your FastAPI code is just an application object. Something has to *run* it and translate raw network bytes into the requests FastAPI understands. That something is an ASGI server, and the default is **Uvicorn**. You will start it like this (details in the next chapter):

```
$ fastapi dev main.py
```

The modern `fastapi` CLI wraps Uvicorn with sensible defaults and auto-reload. Under the hood it is equivalent to `uvicorn main:app --reload`.

2.6 Recap

You verified Python 3.10+, created and activated a virtual environment, installed FastAPI and SQLAlchemy, and learned that Uvicorn is the server that runs your app. You are ready to write code — right after a short but essential detour into the Python features FastAPI depends on.

Chapter 3 — Python You Need: Type Hints and Async

FastAPI's "magic" is really just disciplined use of two standard Python features. Understanding them removes all the mystery.

3.1 Type hints

A type hint annotates a variable, parameter, or return value with its expected type. They do not change how Python runs — Python ignores them at runtime — but tools (and FastAPI) read them.

```
def greet(name: str) -> str:
    return "Hello " + name
```

Here `name: str` says "name is a string" and `-> str` says "this function returns a string." FastAPI reads these annotations to know how to parse and validate data.

Common hints you will use constantly:

```
age: int = 30
price: float = 9.99
is_active: bool = True
tags: list[str] = ["a", "b"]
scores: dict[str, int] = {"math": 90}
```

Optional values

A value that may be `None` is written with `| None`:

```
def find_user(name: str | None = None) -> str:
    ...
```

This means "name is either a string or None, and defaults to None." FastAPI interprets this as an optional parameter.

3.2 Why FastAPI loves type hints

When you write a FastAPI endpoint like this:

```
@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

FastAPI sees `item_id: int` and automatically:

- reads `item_id` from the URL path,
- converts the text `"42"` into the integer `42`,
- returns a clear error if someone passes `"abc"`,
- documents the parameter as an integer in `/docs`.

You wrote one annotation; FastAPI did four jobs.

3.3 Synchronous vs. asynchronous

A normal (synchronous) function runs top to bottom and blocks while waiting. If it calls a database that takes 100ms, the whole worker sits idle for 100ms.

An asynchronous function can *pause* at an `await` point and let the program do other work, then resume. This lets one process handle many concurrent requests efficiently.

```
import asyncio

async def fetch_data():
    print("start")
    await asyncio.sleep(1)  # pauses here, frees the loop
    print("done")
    return 42
```

Key rules:

- `async def` declares a coroutine function.
- `await` can only be used inside an `async def`.
- You `await` things that are themselves awaitable (other async functions, async DB calls, etc.).

3.4 When to use `async def` vs `def` in FastAPI

This trips up beginners, so memorize the rule:

- Use `async def` when your endpoint calls libraries you `await` (async database drivers, `httpx` async client).
- Use plain `def` when your code is synchronous (a regular database call, CPU work, a synchronous library).

FastAPI is smart: if you define an endpoint with plain `def`, it runs it in a thread pool so it does not block the event loop. So **when in doubt, plain `def` is safe**. Never call a blocking, synchronous operation inside an `async def` — that *does* block everything.

3.5 Recap

Type hints are annotations FastAPI reads to validate, parse, and document data. Async functions (`async def` + `await`) let one process handle many waiting requests concurrently. Use `async def` only when you actually `await` something; otherwise plain `def` is safe because FastAPI offloads it to a thread. With these two ideas understood, the framework will never feel like magic again.

Chapter 4 — Your First FastAPI Application

4.1 Hello, world

Create a file `main.py`:

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, FastAPI!"}
```

Three things are happening:

1. `app = FastAPI()` creates the application object. This is what the server runs.
2. `@app.get("/")` is a **decorator** registering the function below it as the handler for HTTP GET requests to the path `/`.
3. The function returns a Python dictionary, which FastAPI automatically converts to a JSON response.

4.2 Running it

```
$ fastapi dev main.py
```

You will see output telling you the server is running at `http://127.0.0.1:8000`. Open that URL in a browser and you will see:

```
{"message": "Hello, FastAPI!"}
```

The `dev` mode enables **auto-reload**: edit and save `main.py`, and the server restarts automatically.

4.3 The interactive docs

Now visit `http://127.0.0.1:8000/docs`. You get the full Swagger UI showing your one endpoint, with a "Try it out" button that sends a real request. Visit `/redoc` for the alternative documentation. You wrote nothing to make these exist.

4.4 Adding more routes and HTTP methods

HTTP methods describe the *intent* of a request. The four you will use most map directly to CRUD:

HTTP method	CRUD operation	Meaning
POST	Create	Make a new resource
GET	Read	Fetch resource(s)
PUT / PATCH	Update	Modify a resource
DELETE	Delete	Remove a resource

Each has a matching decorator:

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, FastAPI!"}

@app.get("/health")
def health_check():
    return {"status": "ok"}

@app.post("/echo")
def echo(payload: dict):
    return {"you_sent": payload}
```

Restart is automatic. In `/docs` you can now POST a JSON body to `/echo` and see it returned.

4.5 What FastAPI did for you

For each route, FastAPI registered the path and method, prepared validation, converted your return value to JSON, set the correct content type, and added the endpoint to the OpenAPI schema powering the docs. You focused only on the logic.

4.6 Recap

You created a `FastAPI()` app, defined routes with method decorators, returned dictionaries that became JSON, and explored the auto-generated docs. You also met the CRUD-to-HTTP-method mapping that structures the rest of the book. Next we make endpoints accept real input.

Chapter 5 — Path Parameters, Query Parameters, and Request Bodies

There are three main ways a client sends data to your API: in the **path**, in the **query string**, and in the **body**. FastAPI handles all three through function parameters and type hints.

5.1 Path parameters

A path parameter is part of the URL itself, used to identify a specific resource.

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/books/{book_id}")
def get_book(book_id: int):
    return {"book_id": book_id}
```

Requesting `/books/5` returns `{"book_id": 5}`. Because you annotated `book_id: int`, FastAPI converts `"5"` to `5` and rejects `/books/abc` with a clear 422 error. The name in the curly braces must match the function parameter name.

Validating path parameters with `Path`

You can add constraints and metadata:

```
from fastapi import FastAPI, Path

app = FastAPI()

@app.get("/books/{book_id}")
def get_book(book_id: int = Path(gt=0, description="The book's ID, must be positive")):
    return {"book_id": book_id}
```

`gt=0` means "greater than 0." Now `/books/0` is rejected automatically.

5.2 Query parameters

Query parameters come after `?` in the URL and are typically used for filtering, sorting, and pagination. Any function parameter that is **not** part of the path becomes a query parameter.

```
@app.get("/books")
def list_books(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}
```

Requesting `/books?skip=20&limit=5` returns `{"skip": 20, "limit": 5}`. Because both have defaults, they are optional. Without defaults, they would be required.

Optional query parameters

```
@app.get("/books")
def list_books(q: str | None = None):
    if q:
        return {"searching_for": q}
    return {"all": True}
```

Validating query parameters with `Query`

```
from fastapi import Query

@app.get("/books")
def list_books(q: str | None = Query(default=None, min_length=3, max_length=50)):
    return {"q": q}
```

Now `q` must be between 3 and 50 characters if provided.

5.3 Request bodies with Pydantic models

For creating and updating resources, clients send a JSON **body**. You describe its shape with a Pydantic model, and FastAPI validates incoming data against it.

```
# main.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class BookCreate(BaseModel):
    title: str
    author: str
    price: float
    in_stock: bool = True

@app.post("/books")
def create_book(book: BookCreate):
    return {"received": book, "message": f"Created '{book.title}'"}
```

When you declare a parameter whose type is a Pydantic model, FastAPI knows it comes from the request body. It will:

- read the JSON body,
- validate every field (rejecting a missing `title` or a non-numeric `price`),
- give you a fully typed `book` object with attribute access (`book.title`),
- document the expected body shape in `/docs`.

Try posting this to `/books`:

```
{"title": "Dune", "author": "Frank Herbert", "price": 18.99}
```

Because `in_stock` has a default, you can omit it.

5.4 Combining all three

FastAPI figures out where each parameter comes from based on its type and whether it appears in the path:

```
@app.put("/books/{book_id}")
def update_book(book_id: int, book: BookCreate, notify: bool = False):
    return {
        "updated_id": book_id,      # from path
        "data": book,               # from body (Pydantic model)
        "notify": notify,          # from query string
    }
```

The rules FastAPI applies:

- If the parameter name is in the path → **path parameter**.

- If the type is a Pydantic model → **request body**.
- Otherwise (a simple type) → **query parameter**.

5.5 Recap

Clients send data in three places. Path parameters identify resources (`/books/{id}`). Query parameters filter and paginate (`?skip=0&limit=10`). Request bodies, described by Pydantic models, carry structured JSON for creating and updating. FastAPI routes each function parameter to the correct source using type hints, and validates all of them automatically. You now have every input mechanism you need to build real endpoints — and Pydantic, which powers bodies, gets a full chapter next.

Part II — Building Real APIs

Chapter 6 — Pydantic: Data Validation Done Right

Pydantic is the data-validation engine inside FastAPI. Mastering it makes everything else easier. The current major version is Pydantic v2, which is what FastAPI uses today.

6.1 What a model is

A Pydantic model is a class that inherits from `BaseModel` and declares fields with type hints. When you create an instance, Pydantic validates and coerces the data, raising a clear error if it does not fit.

```
from pydantic import BaseModel

class Book(BaseModel):
    title: str
    author: str
    price: float
    pages: int

# Valid – note "120" is coerced to int 120
book = Book(title="Dune", author="Herbert", price=18.99, pages="120")
print(book.pages)           # 120 (an int)
print(book.model_dump())    # dict representation
```

`model_dump()` converts the model to a dictionary; `model_dump_json()` converts it to a JSON string. These replace v1's `.dict()` and `.json()`.

6.2 Default values and optional fields

```
class Book(BaseModel):
    title: str
    author: str
    price: float = 0.0           # default, optional
    description: str | None = None # optional, may be null
    tags: list[str] = []         # default empty list
```

A field without a default is **required**. A field with a default is optional.

6.3 Field validation with `Field`

`Field` adds constraints and metadata to a single field:

```
from pydantic import BaseModel, Field

class Book(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    price: float = Field(gt=0, description="Price in USD, must be positive")
    pages: int = Field(ge=1, le=10000)
    isbn: str = Field(pattern=r"^\d{13}$") # exactly 13 digits
```

Common constraints: `gt`, `ge`, `lt`, `le` (numeric bounds), `min_length`, `max_length` (strings and collections), and `pattern` (regex).

6.4 Custom validators

When a rule cannot be expressed with `Field`, write a validator method.

```
from pydantic import BaseModel, field_validator

class Book(BaseModel):
    title: str
    price: float

    @field_validator("title")
    @classmethod
    def title_must_not_be_blank(cls, v: str) -> str:
        if not v.strip():
            raise ValueError("title cannot be blank")
        return v.strip().title() # normalize, e.g. "dune" -> "Dune"
```

For rules that involve more than one field, use `model_validator`:

```
from pydantic import BaseModel, model_validator

class Discount(BaseModel):
    price: float
    sale_price: float

    @model_validator(mode="after")
    def check_sale(self):
        if self.sale_price > self.price:
            raise ValueError("sale_price cannot exceed price")
        return self
```

6.5 Nested models

Models can contain other models, and Pydantic validates the whole tree.

```
class Author(BaseModel):
    name: str
    country: str

class Book(BaseModel):
    title: str
    author: Author          # nested model
    co_authors: list[Author] = []
```

A JSON body like `{"title": "...", "author": {"name": "...", "country": "..."} }` is fully validated, including the nested author.

6.6 The request/response model pattern

A crucial professional habit: use *different* models for input and output. The data a client sends to create a book is not the same as what you return (which includes a generated `id` and timestamps, and excludes secrets like passwords).

```
class BookCreate(BaseModel):      # what the client sends
    title: str
    author: str
    price: float

class BookRead(BaseModel):       # what the API returns
    id: int
    title: str
    author: str
    price: float
```

You will see this pattern everywhere once we reach the database chapters; SQLAlchemy makes it especially clean.

6.7 Recap

Pydantic models declare data shapes with type hints and validate automatically. `Field` adds per-field constraints; `field_validator` and `model_validator` handle custom rules; models nest freely. Separating input models (`...Create`) from output models (`...Read`) is a core professional pattern. With Pydantic understood, we can connect to a real database.

Chapter 7 — SQLModel and Databases

7.1 Why SQLModel

So far our data has lived only in memory and vanished on restart. Real APIs persist data in a database. **SQLModel**, created by the same author as FastAPI, is the perfect bridge: a single class is *both* a Pydantic model *and* a database table (a SQLAlchemy model). You define your data once and use it for validation, serialization, and persistence.

```
$ pip install sqlmodel
```

Under the hood, SQLModel combines **Pydantic** (validation) with **SQLAlchemy** (the mature Python ORM that talks to databases). You get SQLAlchemy's power with Pydantic's ergonomics.

7.2 Core concepts: engine, session, table

Three ideas you must hold in your head:

- **Engine** — the object that knows how to connect to your database. You create one, once, for the whole app.
- **Table model** — a class with `table=True` that maps to a database table. Each instance is a row.
- **Session** — a short-lived workspace for a unit of work. You open one per request, do your reads/writes, commit, and close it.

7.3 Defining a table model

```
# models.py
from sqlmodel import SQLModel, Field

class Book(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    title: str = Field(index=True)
    author: str
    price: float
    in_stock: bool = True
```

Notes:

- `table=True` makes this a real database table.
- `id: int | None ... primary_key=True` is the auto-incrementing primary key. It is `None` before the row is saved and filled in by the database afterward.
- `index=True` creates a database index on `title` for fast lookups.

7.4 Creating the engine and tables

```
# database.py
from sqlmodel import SQLModel, create_engine, Session

DATABASE_URL = "sqlite:///bookstore.db"

# check_same_thread=False is needed only for SQLite with FastAPI
engine = create_engine(DATABASE_URL, echo=True, connect_args={"check_same_thread":
False})

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session
```

- `echo=True` prints the SQL being run — wonderful for learning, turn it off in production.
- `create_db_and_tables()` creates every table for models you have defined.
- `get_session()` is a **generator dependency** (Chapter 10): it yields a session and guarantees it is closed afterward. FastAPI will inject this into endpoints.

7.5 Switching to PostgreSQL later

The beauty of an ORM is that your code barely changes when you switch databases. To move from SQLite to PostgreSQL, install a driver and change the URL:

```
$ pip install psycopg2-binary
```

```
DATABASE_URL = "postgresql://user:password@localhost:5432/bookstore"
# remove the SQLite-only connect_args
engine = create_engine(DATABASE_URL, echo=True)
```

Your models and endpoints stay identical. This is why we develop on SQLite (zero setup) and deploy on PostgreSQL (robust and concurrent).

7.6 The four database operations behind CRUD

Every CRUD endpoint reduces to a small set of session calls:

- **Create:** `session.add(obj)` then `session.commit()` then `session.refresh(obj)`.
- **Read one:** `session.get(Model, id)`.
- **Read many:** `session.exec(select(Model)).all()`.
- **Update:** fetch the object, change attributes, `add`, `commit`, `refresh`.
- **Delete:** fetch the object, `session.delete(obj)`, `commit`.

`refresh` reloads the object from the database so fields the DB filled in (like the new `id`) are populated on your Python object.

7.7 Recap

SQLModel unifies Pydantic and SQLAlchemy so one class is both a validation model and a database table. You create an engine once, define `table=True` models, and use short-lived sessions to read and write. Developing on SQLite and deploying on PostgreSQL is a one-line change. Next we assemble all of this into a complete CRUD API.

Chapter 8 — Building a Complete CRUD API

This is the heart of the book. We build a fully working Bookstore API with all four CRUD operations, proper input/output models, and clean structure. Type every line.

8.1 Project layout

```
bookstore-api/  
├── database.py      # engine + session  
├── models.py        # SQLAlchemy models  
└── main.py          # the FastAPI app and routes
```

8.2 The models

We use SQLAlchemy's pattern of a shared base plus specialized models for the table, creation, reading, and updating.

```
# models.py
from sqlmodel import SQLModel, Field

class BookBase(SQLModel):
    title: str = Field(index=True, min_length=1, max_length=200)
    author: str = Field(min_length=1, max_length=120)
    price: float = Field(gt=0)
    in_stock: bool = True

class Book(BookBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

class BookCreate(BookBase):
    pass

class BookRead(BookBase):
    id: int

class BookUpdate(SQLModel):
    title: str | None = None
    author: str | None = None
    price: float | None = None
    in_stock: bool | None = None
```

Why four models?

- `Book` is the table (rows in the database).
- `BookCreate` is the request body for creating — no `id` (the DB assigns it).
- `BookRead` is the response — includes `id`.
- `BookUpdate` makes every field optional, so clients can send only what they want to change (a partial update).

8.3 The database module

```
# database.py
from sqlmodel import SQLModel, create_engine, Session

DATABASE_URL = "sqlite:///bookstore.db"
engine = create_engine(DATABASE_URL, echo=True, connect_args={"check_same_thread":
False})

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session
```

8.4 The application with all CRUD routes

```

# main.py
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException, Depends, Query, status
from sqlmodel import Session, select

from database import engine, create_db_and_tables, get_session
from models import Book, BookCreate, BookRead, BookUpdate

@asynccontextmanager
async def lifespan(app: FastAPI):
    # runs on startup
    create_db_and_tables()
    yield
    # runs on shutdown (nothing needed here)

app = FastAPI(title="Bookstore API", lifespan=lifespan)

# ----- CREATE -----
@app.post("/books", response_model=BookRead, status_code=status.HTTP_201_CREATED)
def create_book(book: BookCreate, session: Session = Depends(get_session)):
    db_book = Book.model_validate(book) # convert input model -> table model
    session.add(db_book)
    session.commit()
    session.refresh(db_book)           # load the DB-generated id
    return db_book

# ----- READ MANY -----
@app.get("/books", response_model=list[BookRead])
def list_books(
    offset: int = 0,
    limit: int = Query(default=10, le=100),
    session: Session = Depends(get_session),
):
    books = session.exec(select(Book).offset(offset).limit(limit)).all()
    return books

# ----- READ ONE -----
@app.get("/books/{book_id}", response_model=BookRead)
def get_book(book_id: int, session: Session = Depends(get_session)):
    book = session.get(Book, book_id)
    if not book:
        raise HTTPException(status_code=404, detail="Book not found")
    return book

# ----- UPDATE -----

```

```

@app.patch("/books/{book_id}", response_model=BookRead)
def update_book(book_id: int, book: BookUpdate, session: Session = Depends(get_session))
:
    db_book = session.get(Book, book_id)
    if not db_book:
        raise HTTPException(status_code=404, detail="Book not found")
    book_data = book.model_dump(exclude_unset=True) # only fields the client sent
    for key, value in book_data.items():
        setattr(db_book, key, value)
    session.add(db_book)
    session.commit()
    session.refresh(db_book)
    return db_book

# ----- DELETE -----
@app.delete("/books/{book_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_book(book_id: int, session: Session = Depends(get_session)):
    book = session.get(Book, book_id)
    if not book:
        raise HTTPException(status_code=404, detail="Book not found")
    session.delete(book)
    session.commit()

```

8.5 Reading the code carefully

A few details are worth dwelling on because they recur in every project:

- `response_model=BookRead` tells FastAPI to filter and shape the output. Even though we return a `Book` table object, FastAPI serializes it through `BookRead`, guaranteeing the response shape and hiding anything not in `BookRead`.
- `status_code=201` on create and `204` on delete follow HTTP conventions: 201 Created, 204 No Content.
- `Book.model_validate(book)` converts the incoming `BookCreate` into a `Book` table instance.
- `exclude_unset=True` is the key to partial updates: it returns only the fields the client actually sent, so unspecified fields keep their current values.
- `Depends(get_session)` injects a fresh database session per request. We unpack dependency injection fully in Chapter 10.
- `lifespan` is the modern way to run startup/shutdown code (here, creating tables). It replaces the older `@app.on_event("startup")`.

8.6 Running and testing it

```
$ fastapi dev main.py
```

Open `/docs` and exercise every endpoint:

1. **POST** `/books` with `{"title": "Dune", "author": "Frank Herbert", "price": 18.99}` → 201, returns the book with an `id`.
2. **GET** `/books` → list containing your book.
3. **GET** `/books/1` → that single book.
4. **PATCH** `/books/1` with `{"price": 15.00}` → price updated, everything else unchanged.
5. **DELETE** `/books/1` → 204, then **GET** `/books/1` returns 404.

You have just built a complete, persistent, validated, self-documented REST API.

8.7 Recap

Using four cooperating models (table, create, read, update), a session dependency, and the five standard routes, you built a full CRUD API backed by a real database. You learned the role of `response_model`, proper status codes, partial updates with `exclude_unset`, and the `lifespan` startup hook. Everything from here extends this foundation.

Chapter 9 — Relationships Between Models

Real data is connected: an author writes many books; a book belongs to an author; an order contains many items. SQLAlchemy models these relationships with foreign keys and `Relationship` attributes.

9.1 One-to-many: Author and Books

An author has many books; each book has one author.

```
# models.py
from sqlalchemy import SQLModel, Field, Relationship

class Author(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    books: list["Book"] = Relationship(back_populates="author")

class Book(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    title: str
    author_id: int | None = Field(default=None, foreign_key="author.id")
    author: Author | None = Relationship(back_populates="books")
```

Two mechanisms work together:

- `foreign_key="author.id"` stores the link in the database — each book row records which author it belongs to.
- `Relationship(back_populates=...)` is Python-side convenience: `author.books` gives the list of books, and `book.author` gives the author object. `back_populates` keeps both sides in sync.

9.2 Creating related rows

```
with Session(engine) as session:
    herbert = Author(name="Frank Herbert")
    dune = Book(title="Dune", author=herbert)           # link by object
    children = Book(title="Children of Dune", author=herbert)
    session.add(herbert)    # adding the author cascades to its books
    session.commit()
    session.refresh(herbert)
    print(herbert.books)    # [Dune, Children of Dune]
```

9.3 Returning nested data with response models

To return an author together with their books, define read models that nest:

```
class BookRead(SQLModel):
    id: int
    title: str

class AuthorReadWithBooks(SQLModel):
    id: int
    name: str
    books: list[BookRead] = []
```

```
@app.get("/authors/{author_id}", response_model=AuthorReadWithBooks)
def get_author(author_id: int, session: Session = Depends(get_session)):
    author = session.get(Author, author_id)
    if not author:
        raise HTTPException(404, "Author not found")
    return author
```

FastAPI serializes the author and walks the `books` relationship to embed each book.

9.4 Many-to-many: Books and Tags

A book can have many tags; a tag applies to many books. This needs a **link table**.

```
class BookTagLink(SQLModel, table=True):
    book_id: int | None = Field(default=None, foreign_key="book.id", primary_key=True)
    tag_id: int | None = Field(default=None, foreign_key="tag.id", primary_key=True)

class Tag(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    books: list["Book"] = Relationship(back_populates="tags", link_model=BookTagLink)

class Book(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    title: str
    tags: list[Tag] = Relationship(back_populates="books", link_model=BookTagLink)
```

The `link_model` connects the two sides through the intermediary table.

9.5 A note on the N+1 problem

When you load 100 authors and then access `.books` on each, a naive ORM issues 1 query for the authors plus 100 more for the books — the "N+1 problem." For performance-critical lists, eagerly load relationships:

```
from sqlalchemy.orm import selectinload

statement = select(Author).options(selectinload(Author.books))
authors = session.exec(statement).all()
```

This fetches all related books in one extra query instead of N.

9.6 Recap

Relationships use a `foreign_key` field (stored in the DB) plus `Relationship(back_populates=...)` (Python convenience). One-to-many links an author to many books; many-to-many uses a link table with `link_model`. Nested response models embed related data in responses. Watch for the N+1 problem and use `selectinload` for hot paths.

Chapter 10 — Dependency Injection

Dependency injection (DI) is one of FastAPI's most powerful and elegant features. You already used it with `Depends(get_session)`. Now we understand it fully.

10.1 The idea

A "dependency" is a function whose result FastAPI computes and supplies to your endpoint. Instead of an endpoint creating everything it needs, it *declares* what it needs, and FastAPI provides it. This makes code reusable, testable, and clean.

```
from fastapi import Depends

def pagination(skip: int = 0, limit: int = 10) -> dict:
    return {"skip": skip, "limit": limit}

@app.get("/books")
def list_books(page: dict = Depends(pagination)):
    return page
```

Now `skip` and `limit` are read, validated, and documented exactly as if they were declared on the endpoint — but the logic lives in one reusable place. Any endpoint can reuse `pagination`.

10.2 Dependencies with cleanup (`yield`)

A dependency can `yield` a value and run teardown code afterward. This is exactly how `get_session` guarantees the session closes even if the endpoint raises an error.

```
def get_session():
    session = Session(engine)
    try:
        yield session          # provided to the endpoint
    finally:
        session.close()       # always runs, even on error
```

The code before `yield` runs before the request; the code after `yield` runs after the response is sent.

10.3 Sub-dependencies

Dependencies can depend on other dependencies, forming a tree. FastAPI resolves the whole tree for you.

```
def get_session():
    with Session(engine) as session:
        yield session

def get_book_or_404(book_id: int, session: Session = Depends(get_session)) -> Book:
    book = session.get(Book, book_id)
    if not book:
        raise HTTPException(404, "Book not found")
    return book

@app.get("/books/{book_id}", response_model=BookRead)
def read_book(book: Book = Depends(get_book_or_404)):
    return book
```

The endpoint is now trivial — the "fetch or 404" logic is a reusable dependency that itself depends on the session. This pattern eliminates repetition across your read, update, and delete routes.

10.4 Class-based dependencies

A class can be a dependency; FastAPI calls it and injects the instance. This is handy for grouping related parameters.

```
class CommonParams:
    def __init__(self, q: str | None = None, skip: int = 0, limit: int = 10):
        self.q = q
        self.skip = skip
        self.limit = limit

@app.get("/books")
def list_books(params: CommonParams = Depends(CommonParams)):
    return {"q": params.q, "skip": params.skip, "limit": params.limit}
```

10.5 The **Annotated** style (recommended)

Modern FastAPI prefers **Annotated** for dependencies because it is reusable and type-checker friendly:

```

from typing import Annotated
from fastapi import Depends

SessionDep = Annotated[Session, Depends(get_session)]

@app.get("/books/{book_id}", response_model=BookRead)
def get_book(book_id: int, session: SessionDep):
    book = session.get(Book, book_id)
    if not book:
        raise HTTPException(404, "Book not found")
    return book

```

Defining `SessionDep` once and reusing it everywhere keeps signatures short and consistent.

10.6 Application-wide and router-level dependencies

You can attach dependencies to whole groups of routes — useful for auth checks that must run on every endpoint in a section:

```

app = FastAPI(dependencies=[Depends(verify_api_key)])    # every route
# or per router:
router = APIRouter(dependencies=[Depends(verify_api_key)])

```

10.7 Recap

Dependency injection lets endpoints declare what they need while FastAPI supplies it. `yield` dependencies handle setup/teardown (like sessions). Dependencies nest into trees, can be classes, and are best expressed with `Annotated`. Group-level dependencies enforce cross-cutting concerns like authentication. DI is the backbone of clean, testable FastAPI applications.

Chapter 11 — Error Handling and Validation

A professional API fails *clearly*. Clients should always receive a meaningful status code and message. FastAPI gives you several layers of control.

11.1 Raising HTTP errors

The everyday tool is `HTTPException`:

```
from fastapi import HTTPException

@app.get("/books/{book_id}")
def get_book(book_id: int, session: SessionDep):
    book = session.get(Book, book_id)
    if not book:
        raise HTTPException(
            status_code=404,
            detail="Book not found",
            headers={"X-Error": "BookMissing"}, # optional custom headers
        )
    return book
```

`detail` can be a string or any JSON-serializable object. FastAPI returns it as `{"detail": ...}` with the chosen status code.

11.2 Choosing the right status code

A quick mental model:

- **400 Bad Request** — the client sent something semantically wrong you detected yourself.
- **401 Unauthorized** — not authenticated (no/invalid credentials).
- **403 Forbidden** — authenticated but not allowed.
- **404 Not Found** — the resource does not exist.
- **409 Conflict** — the request conflicts with current state (e.g., duplicate).
- **422 Unprocessable Entity** — validation failed (FastAPI raises this automatically).
- **500 Internal Server Error** — an unhandled exception on your side.

11.3 How automatic validation errors look

When incoming data fails Pydantic validation, FastAPI returns **422** with a precise, machine-readable body pointing to each bad field:

```
{
  "detail": [
    {
      "type": "greater_than",
      "loc": ["body", "price"],
      "msg": "Input should be greater than 0",
      "input": -5
    }
  ]
}
```

`loc` tells the client exactly where the problem is (here, `price` in the body). You did not write any of this — it comes from your type hints and `Field` constraints.

11.4 Custom exception handlers

For domain-specific errors, define your own exception and a handler that turns it into a response. This keeps endpoint code clean.

```
from fastapi import Request
from fastapi.responses import JSONResponse

class OutOfStockError(Exception):
    def __init__(self, book_id: int):
        self.book_id = book_id

@app.exception_handler(OutOfStockError)
async def out_of_stock_handler(request: Request, exc: OutOfStockError):
    return JSONResponse(
        status_code=409,
        content={"detail": f"Book {exc.book_id} is out of stock"},
    )

@app.post("/orders")
def place_order(book_id: int, session: SessionDep):
    book = session.get(Book, book_id)
    if not book or not book.in_stock:
        raise OutOfStockError(book_id)  # handler turns this into a 409
    return {"ordered": book_id}
```

11.5 Overriding the default validation error format

If you want validation errors in your own shape, override the built-in handler:

```
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse

@app.exception_handler(RequestValidationError)
async def validation_handler(request, exc):
    return JSONResponse(
        status_code=422,
        content={"errors": exc.errors(), "body": exc.body},
    )
```

11.6 A reusable "get or 404" helper

Combining this chapter with DI, here is the pattern you will use constantly:

```
def get_book_or_404(book_id: int, session: SessionDep) -> Book:
    book = session.get(Book, book_id)
    if not book:
        raise HTTPException(404, f"Book {book_id} not found")
    return book
```

11.7 Recap

Use `HTTPException` for routine errors and pick status codes deliberately. FastAPI emits detailed 422 responses automatically from your validation rules. Custom exceptions plus `@app.exception_handler` keep domain errors out of your endpoint bodies, and you can override the default formats when you need a house style. Clear, consistent errors are what separate a toy API from a professional one.

Part III — Production-Grade Features

Chapter 12 — Authentication and Authorization with JWT

Almost every real API must answer two questions: *who are you* (authentication) and *what are you allowed to do* (authorization). We implement the modern, stateless standard: JSON Web Tokens.

12.1 The concepts

- **Authentication** verifies identity (login).
- **Authorization** decides permissions (roles, ownership).
- A **JWT** is a signed token the server issues at login. The client sends it on every subsequent request (in the `Authorization: Bearer <token>` header). Because it is signed with a secret, the server can trust it without storing session state.
- Passwords are never stored in plain text; we store a one-way **hash**.

```
$ pip install "passlib[bcrypt]" "python-jose[cryptography]"
```

12.2 Hashing passwords

```
# security.py
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(plain: str) -> str:
    return pwd_context.hash(plain)

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)
```

bcrypt is deliberately slow and salts each hash, making stolen password databases far harder to crack.

12.3 Creating and decoding tokens

```
# security.py (continued)
from datetime import datetime, timedelta, timezone
from jose import jwt, JWTError

SECRET_KEY = "change-me-to-a-long-random-string"  # load from env in production!
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def decode_token(token: str) -> dict | None:
    try:
        return jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    except JWTError:
        return None
```

12.4 The User model and registration

```
# models.py
from sqlmodel import SQLModel, Field

class User(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    username: str = Field(index=True, unique=True)
    hashed_password: str
    role: str = "user"  # "user" or "admin"

class UserCreate(SQLModel):
    username: str
    password: str

class UserRead(SQLModel):
    id: int
    username: str
    role: str
```

```

@app.post("/register", response_model=UserRead, status_code=201)
def register(data: UserCreate, session: SessionDep):
    existing = session.exec(select(User).where(User.username == data.username)).first()
    if existing:
        raise HTTPException(409, "Username already taken")
    user = User(username=data.username, hashed_password=hash_password(data.password))
    session.add(user)
    session.commit()
    session.refresh(user)
    return user

```

12.5 The login endpoint

FastAPI integrates with OAuth2's password flow, which [/docs](#) understands natively (you get a login button).

```

from fastapi.security import OAuth2PasswordRequestForm

@app.post("/token")
def login(form: OAuth2PasswordRequestForm = Depends(), session: SessionDep = None):
    user = session.exec(select(User).where(User.username == form.username)).first()
    if not user or not verify_password(form.password, user.hashed_password):
        raise HTTPException(401, "Incorrect username or password")
    token = create_access_token({"sub": user.username, "role": user.role})
    return {"access_token": token, "token_type": "bearer"}

```

12.6 Protecting routes with a current-user dependency

This is where DI shines. We write one dependency that extracts and validates the token, then reuse it everywhere.

```

from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

def get_current_user(token: str = Depends(oauth2_scheme), session: SessionDep = None) -
> User:
    payload = decode_token(token)
    if payload is None:
        raise HTTPException(401, "Invalid or expired token")
    username = payload.get("sub")
    user = session.exec(select(User).where(User.username == username)).first()
    if user is None:
        raise HTTPException(401, "User not found")
    return user

@app.get("/me", response_model=UserRead)
def read_me(current_user: User = Depends(get_current_user)):
    return current_user

```

Any endpoint that adds `current_user: User = Depends(get_current_user)` now requires a valid token automatically. If the token is missing or invalid, the request never reaches your code — it gets a 401.

12.7 Role-based authorization

Build authorization as a dependency layered on top of authentication:

```

def require_admin(current_user: User = Depends(get_current_user)) -> User:
    if current_user.role != "admin":
        raise HTTPException(403, "Admin privileges required")
    return current_user

@app.delete("/books/{book_id}", status_code=204)
def delete_book(book_id: int, session: SessionDep, admin: User = Depends(require_admin))
:
    book = session.get(Book, book_id)
    if not book:
        raise HTTPException(404, "Book not found")
    session.delete(book)
    session.commit()

```

Now only admins can delete books — enforced declaratively, in one line.

12.8 Recap

JWT auth is stateless: log in once, receive a signed token, send it on every request. Hash passwords with `bcrypt`, sign tokens with a secret, and validate them in a `get_current_user` dependency you reuse across protected routes. Layer authorization (like `require_admin`) on top. Keep the secret key in environment variables, never in source. This is the same pattern that secures production APIs everywhere.

Chapter 13 — Async Databases and Performance

You chose an async-capable stack, so let us use it properly. Async databases let one process serve many requests that are waiting on the database, dramatically improving throughput under load.

13.1 When async actually helps

Async shines for **I/O-bound** work: waiting on a database, an external API, or the network. It does *not* speed up **CPU-bound** work (image processing, heavy computation) — for that you need processes, not async. An API that mostly reads and writes a database is the ideal async candidate.

13.2 Installing an async driver

PostgreSQL with the async `asyncpg` driver is the standard production choice:

```
$ pip install asyncpg
```

The URL uses the `+asyncpg` dialect:

```
DATABASE_URL = "postgresql+asyncpg://user:password@localhost/bookstore"
```

For async SQLite during development:

```
$ pip installaiosqlite
```

```
DATABASE_URL = "sqlite+aiosqlite:///bookstore.db"
```

13.3 The async engine and session

```
# database.py
from sqlmodel import SQLModel
from sqlmodel.ext.asyncio.session import AsyncSession
from sqlalchemy.ext.asyncio import create_async_engine
from sqlalchemy.orm import sessionmaker

engine = create_async_engine(DATABASE_URL, echo=True)

async_session = sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

async def create_db_and_tables():
    async with engine.begin() as conn:
        await conn.run_sync(SQLModel.metadata.create_all)

async def get_session() -> AsyncSession:
    async with async_session() as session:
        yield session
```

Note `expire_on_commit=False`: it keeps objects usable after commit, which matters because we serialize them into the response after committing.

13.4 Async CRUD endpoints

Every database call is now awaited, and endpoints are `async def`:

```

from typing import Annotated
from fastapi import FastAPI, Depends, HTTPException
from sqlmodel import select
from sqlmodel.ext.asyncio.session import AsyncSession

SessionDep = Annotated[AsyncSession, Depends(get_session)]
app = FastAPI()

@app.post("/books", response_model=BookRead, status_code=201)
async def create_book(book: BookCreate, session: SessionDep):
    db_book = Book.model_validate(book)
    session.add(db_book)
    await session.commit()
    await session.refresh(db_book)
    return db_book

@app.get("/books", response_model=list[BookRead])
async def list_books(session: SessionDep):
    result = await session.exec(select(Book))
    return result.all()

@app.get("/books/{book_id}", response_model=BookRead)
async def get_book(book_id: int, session: SessionDep):
    book = await session.get(Book, book_id)
    if not book:
        raise HTTPException(404, "Book not found")
    return book

```

The shape is identical to the synchronous version from Chapter 8 — just `async` / `await` added. This symmetry is intentional and makes migrating easy.

13.5 Performance practices that matter more than async

Async is not a silver bullet. These often matter more:

- **Indexes.** Add `index=True` to columns you filter or sort by. A missing index turns a fast lookup into a full table scan.
- **Pagination.** Never return unbounded lists. Always `offset / limit` and cap the maximum.
- **Avoid N+1.** Eager-load relationships (Chapter 9) on list endpoints.
- **Connection pooling.** The engine pools connections by default; tune `pool_size` for your workload.

- **Caching.** For expensive, rarely changing reads, cache with Redis in front of the database.

13.6 Recap

Async databases boost throughput for I/O-bound APIs by letting one process serve many waiting requests. Use an async driver (`asyncpg` / `aiosqlite`), an async engine and session, and `await` every DB call inside `async def` endpoints. But remember the fundamentals — indexes, pagination, avoiding N+1, and caching — frequently deliver bigger wins than async alone.

Chapter 14 — Background Tasks and WebSockets

Not all work fits the request/response model. Sometimes you want to respond immediately and do work afterward; sometimes you need a live, two-way connection.

14.1 Background tasks

A background task runs *after* the response is sent. It is perfect for fire-and-forget work like sending a confirmation email, where the client should not wait.

```
from fastapi import BackgroundTasks

def write_log(message: str):
    with open("log.txt", "a") as f:
        f.write(message + "\n")

@app.post("/books", response_model=BookRead, status_code=201)
def create_book(book: BookCreate, background_tasks: BackgroundTasks, session: SessionDep):
    db_book = Book.model_validate(book)
    session.add(db_book)
    session.commit()
    session.refresh(db_book)
    background_tasks.add_task(write_log, f"Created book {db_book.id}")
    return db_book  # response returns now; the log write happens after
```

The client gets its `201` immediately; `write_log` runs in the background.

When to graduate to a task queue

`BackgroundTasks` runs in the same process, so it suits light work. For heavy, retryable, or scheduled jobs (processing uploads, generating reports, sending thousands of emails), use a dedicated task queue like **Celery** or **ARQ** with a broker such as Redis. The API enqueues a job and a separate worker process handles it.

14.2 WebSockets

HTTP is request/response. **WebSockets** open a persistent, bidirectional channel — ideal for chat, live notifications, dashboards, and collaborative editing.

```

from fastapi import WebSocket, WebSocketDisconnect

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    try:
        while True:
            data = await websocket.receive_text()
            await websocket.send_text(f"You said: {data}")
    except WebSocketDisconnect:
        print("Client disconnected")

```

The flow: `accept()` the connection, then loop, awaiting incoming messages and sending replies, until the client disconnects.

A simple broadcast manager

For something like notifications to many clients, track connections and broadcast:

```

class ConnectionManager:
    def __init__(self):
        self.active: list[WebSocket] = []

    async def connect(self, ws: WebSocket):
        await ws.accept()
        self.active.append(ws)

    def disconnect(self, ws: WebSocket):
        self.active.remove(ws)

    async def broadcast(self, message: str):
        for connection in self.active:
            await connection.send_text(message)

manager = ConnectionManager()

@app.websocket("/notifications")
async def notifications(websocket: WebSocket):
    await manager.connect(websocket)
    try:
        while True:
            msg = await websocket.receive_text()
            await manager.broadcast(msg)
    except WebSocketDisconnect:
        manager.disconnect(websocket)

```

14.3 Recap

Background tasks let you respond first and do light work afterward; graduate to Celery/ARQ for heavy jobs. WebSockets provide persistent two-way communication for real-time features, with a simple accept/loop pattern and an optional connection manager for broadcasting. Both expand FastAPI well beyond plain request/response.

Chapter 15 — Testing Your API

Untested code is code you are afraid to change. FastAPI makes testing genuinely pleasant with a built-in test client and clean dependency overrides.

15.1 Tools

```
$ pip install pytest httpx
```

FastAPI provides `TestClient` (built on `httpx`) that calls your app in-process — no running server needed, and tests are fast.

15.2 A first test

```
# test_main.py
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}
```

Run it:

```
$ pytest -v
```

15.3 Isolating the database with dependency overrides

Tests must not touch your real database. Override the session dependency to use a temporary in-memory SQLite database. This is the single most important testing technique in FastAPI.

```

# test_books.py
import pytest
from fastapi.testclient import TestClient
from sqlmodel import SQLModel, Session, create_engine
from sqlmodel.pool import StaticPool

from main import app
from database import get_session

@pytest.fixture(name="session")
def session_fixture():
    engine = create_engine(
        "sqlite://", # in-memory
        connect_args={"check_same_thread": False},
        poolclass=StaticPool,
    )
    SQLModel.metadata.create_all(engine)
    with Session(engine) as session:
        yield session

@pytest.fixture(name="client")
def client_fixture(session):
    def get_session_override():
        return session

    app.dependency_overrides[get_session] = get_session_override
    client = TestClient(app)
    yield client
    app.dependency_overrides.clear()

```

`app.dependency_overrides` swaps the real `get_session` for the test one *only during tests*. Each test gets a fresh, empty database.

15.4 Testing the full CRUD cycle

```
def test_create_and_read_book(client):
    # CREATE
    payload = {"title": "Dune", "author": "Frank Herbert", "price": 18.99}
    r = client.post("/books", json=payload)
    assert r.status_code == 201
    data = r.json()
    assert data["title"] == "Dune"
    assert "id" in data
    book_id = data["id"]

    # READ
    r = client.get(f"/books/{book_id}")
    assert r.status_code == 200
    assert r.json()["author"] == "Frank Herbert"

def test_update_book(client):
    created = client.post("/books", json={"title": "X", "author": "Y", "price":
5}).json()
    r = client.patch(f"/books/{created['id']}", json={"price": 9.99})
    assert r.status_code == 200
    assert r.json()["price"] == 9.99

def test_delete_book(client):
    created = client.post("/books", json={"title": "X", "author": "Y", "price":
5}).json()
    assert client.delete(f"/books/{created['id']}").status_code == 204
    assert client.get(f"/books/{created['id']}").status_code == 404

def test_validation_error(client):
    r = client.post("/books", json={"title": "X", "author": "Y", "price": -1})
    assert r.status_code == 422 # price must be > 0
```

15.5 Testing protected endpoints

Authenticate, capture the token, and send it as a header:

```
def test_protected_route(client):
    client.post("/register", json={"username": "alice", "password": "secret"})
    token = client.post(
        "/token", data={"username": "alice", "password": "secret"}
    ).json()["access_token"]

    r = client.get("/me", headers={"Authorization": f"Bearer {token}"})
    assert r.status_code == 200
    assert r.json()["username"] == "alice"
```

15.6 Recap

`TestClient` runs your app in-process for fast, server-free tests. The key technique is overriding the session dependency with an in-memory database so tests are isolated and repeatable. Test each CRUD operation, the validation failures, and your authenticated routes. With a test suite in place, you can refactor and add features confidently.

Chapter 16 — Middleware, CORS, and Configuration

The finishing touches that make an API deployable: cross-origin support, request-wide behavior, and clean configuration.

16.1 CORS

Browsers block a web page on one origin (e.g., `https://myapp.com`) from calling an API on another (e.g., `https://api.myapp.com`) unless the API explicitly allows it. This is **CORS** (Cross-Origin Resource Sharing). If your frontend cannot reach your API from the browser, CORS is almost always the reason.

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://myapp.com", "http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

In production, list specific origins — avoid `allow_origins=["*"]` when credentials are involved.

16.2 Middleware

Middleware wraps every request and response, perfect for cross-cutting concerns like logging, timing, or adding headers.

```
import time
from fastapi import Request

@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)  # run the rest of the app
    response.headers["X-Process-Time"] = str(time.perf_counter() - start)
    return response
```

The pattern is always: do something before, call `await call_next(request)`, do something after, return the response.

16.3 Configuration with environment variables

Never hardcode secrets or environment-specific values. Use Pydantic's settings management, which reads from environment variables and `.env` files with validation.

```
$ pip install pydantic-settings
```

```
# config.py
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    database_url: str = "sqlite:///bookstore.db"
    secret_key: str
    access_token_expire_minutes: int = 30

    model_config = SettingsConfigDict(env_file=".env")

settings = Settings()
```

A `.env` file (never commit it — add to `.gitignore`):

```
SECRET_KEY=a-very-long-random-secret
DATABASE_URL=postgresql://user:pass@localhost/bookstore
```

Now use `settings.secret_key` anywhere. Values come from the environment in production and from `.env` in development, all validated by Pydantic.

16.4 Organizing large apps with routers

As your app grows, splitting routes into modules with `APIRouter` keeps it maintainable.

```
# routers/books.py
from fastapi import APIRouter

router = APIRouter(prefix="/books", tags=["books"])

@router.get("")
def list_books():
    ...

@router.get("/{book_id}")
def get_book(book_id: int):
    ...
```

```
# main.py
from fastapi import FastAPI
from routers import books, users

app = FastAPI()
app.include_router(books.router)
app.include_router(users.router)
```

`prefix` prepends the path; `tags` groups endpoints in `/docs`. This is how production apps stay organized across dozens of endpoints.

16.5 Recap

CORS middleware lets browsers from approved origins call your API. Custom middleware handles cross-cutting concerns around every request. `pydantic-settings` keeps configuration and secrets in environment variables, validated and typed. `APIRouter` splits a growing app into clean modules. These are the structural pieces every production deployment needs.

Chapter 17 — Deployment

Your API works locally. Now make it run reliably on a server.

17.1 Production server settings

In development you used `fastapi dev`. For production, run without reload and with multiple workers, or use `fastapi run`:

```
$ fastapi run main.py
```

Under the hood this runs Uvicorn. For multiple worker processes (to use multiple CPU cores), a common production setup runs Uvicorn workers managed by Gunicorn, or simply:

```
$ uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4
```

A good rule of thumb is `(2 × CPU cores) + 1` workers. Each worker is a separate process serving requests concurrently.

17.2 Containerizing with Docker

Docker packages your app and its dependencies into a portable image that runs identically everywhere.

```
# Dockerfile
FROM python:3.12-slim

WORKDIR /code

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["fastapi", "run", "main.py", "--port", "8000"]
```

Build and run:

```
$ docker build -t bookstore-api .
$ docker run -p 8000:8000 --env-file .env bookstore-api
```

17.3 Running with a database via Docker Compose

For local production-like setups, Compose runs your API and PostgreSQL together:

```
# docker-compose.yml
services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://postgres:postgres@db:5432/bookstore
      SECRET_KEY: ${SECRET_KEY}
    depends_on:
      - db
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: bookstore
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

```
$ docker compose up --build
```

17.4 Database migrations with Alembic

`SQLModel.metadata.create_all` creates tables but cannot evolve them safely. In production, schema changes go through **Alembic**, which generates and applies versioned migrations.

```
$ pip install alembic
$ alembic init alembic
# configure the URL and target metadata, then:
$ alembic revision --autogenerate -m "add books table"
$ alembic upgrade head
```

This keeps your database schema in version control and applies changes predictably across environments.

17.5 Production checklist

Before going live, confirm:

- `DEBUG` / `echo` is off and tracebacks are not exposed to clients.
- Secrets come from environment variables, never source code.
- HTTPS is enforced (usually terminated by a reverse proxy like Nginx or a cloud load balancer).
- CORS lists only trusted origins.
- A real database (PostgreSQL) with connection pooling is used.
- Logging and a health-check endpoint (`/health`) are in place for monitoring.
- Rate limiting protects against abuse (e.g., via `slowapi` or a gateway).
- Migrations are managed by Alembic.

17.6 Where to host

FastAPI runs anywhere Python or Docker runs: managed platforms (Render, Railway, Fly.io, Heroku), serverless (AWS Lambda via Mangum), or full control on a VM/Kubernetes (AWS, GCP, Azure, DigitalOcean). Start simple — a managed platform with a managed PostgreSQL database — and scale up only when you need to.

17.7 Recap

Run production with multiple Uvicorn workers, package the app in Docker for portable, reproducible deploys, and use Docker Compose to pair it with PostgreSQL. Manage schema changes with Alembic, lock down secrets and CORS, enforce HTTPS, and add logging, health checks, and rate limiting. Then pick a host that matches your scale.

Chapter 18 — Capstone: The Complete Bookstore API

This final chapter assembles everything into one coherent, production-shaped project. Use it as a reference architecture.

18.1 Final project structure

```
bookstore-api/
├── app/
│   ├── __init__.py
│   ├── main.py           # app factory, routers, middleware
│   ├── config.py         # pydantic-settings
│   ├── database.py       # engine + session dependency
│   ├── models.py        # SQLAlchemy models (table + schemas)
│   ├── security.py       # hashing + JWT
│   ├── deps.py           # shared dependencies (session, current user)
│   └── routers/
│       ├── auth.py       # register, token, me
│       └── books.py       # CRUD for books
├── tests/
│   ├── conftest.py       # fixtures with in-memory DB override
│   └── test_books.py
├── .env                  # secrets (gitignored)
├── requirements.txt
├── Dockerfile
└── docker-compose.yml
```

18.2 Shared dependencies

```
# app/deps.py
from typing import Annotated
from fastapi import Depends, HTTPException
from sqlmodel import Session, select

from .database import engine
from .security import decode_token
from .models import User
from fastapi.security import OAuth2PasswordBearer

def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/token")

def get_current_user(token: Annotated[str, Depends(oauth2_scheme)], session:
SessionDep) -> User:
    payload = decode_token(token)
    if not payload:
        raise HTTPException(401, "Invalid token")
    user = session.exec(select(User).where(User.username == payload.get("sub"))).first()
    if not user:
        raise HTTPException(401, "User not found")
    return user

CurrentUser = Annotated[User, Depends(get_current_user)]

def require_admin(user: CurrentUser) -> User:
    if user.role != "admin":
        raise HTTPException(403, "Admin only")
    return user
```

18.3 The books router (full CRUD, protected writes)

```

# app/routers/books.py
from fastapi import APIRouter, HTTPException, Query, status
from sqlmodel import select

from ..deps import SessionDep, CurrentUser, require_admin
from ..models import Book, BookCreate, BookRead, BookUpdate
from fastapi import Depends

router = APIRouter(prefix="/books", tags=["books"])

@router.post("", response_model=BookRead, status_code=status.HTTP_201_CREATED)
def create_book(book: BookCreate, session: SessionDep, user: CurrentUser):
    db_book = Book.model_validate(book)
    session.add(db_book)
    session.commit()
    session.refresh(db_book)
    return db_book

@router.get("", response_model=list[BookRead])
def list_books(session: SessionDep, offset: int = 0, limit: int = Query(10, le=100)):
    return session.exec(select(Book).offset(offset).limit(limit)).all()

@router.get("/{book_id}", response_model=BookRead)
def get_book(book_id: int, session: SessionDep):
    book = session.get(Book, book_id)
    if not book:
        raise HTTPException(404, "Book not found")
    return book

@router.patch("/{book_id}", response_model=BookRead)
def update_book(book_id: int, data: BookUpdate, session: SessionDep, user: CurrentUser):
    book = session.get(Book, book_id)
    if not book:
        raise HTTPException(404, "Book not found")
    for key, value in data.model_dump(exclude_unset=True).items():
        setattr(book, key, value)
    session.add(book)
    session.commit()
    session.refresh(book)
    return book

@router.delete("/{book_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_book(book_id: int, session: SessionDep, admin=Depends(require_admin)):
    book = session.get(Book, book_id)
    if not book:
        raise HTTPException(404, "Book not found")

```

```
session.delete(book)
session.commit()
```

18.4 Wiring it together

```
# app/main.py
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from sqlmodel import SQLModel

from .database import engine
from .routers import books, auth

@asynccontextmanager
async def lifespan(app: FastAPI):
    SQLModel.metadata.create_all(engine)
    yield

app = FastAPI(title="Bookstore API", version="1.0.0", lifespan=lifespan)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],      # tighten in production
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(auth.router)
app.include_router(books.router)

@app.get("/health", tags=["system"])
def health():
    return {"status": "ok"}
```

18.5 What you have built

This project demonstrates every concept in the book working together: SQLModel models with separate create/read/update schemas, full CRUD with correct status codes, JWT authentication and role-based authorization, dependency injection for sessions and the current user, validation and clear errors, CORS, a health check, the lifespan startup hook, modular routers, environment-based configuration, a test suite with an isolated database, and a Docker-based deployment. From here you

can add features — orders, reviews, search, pagination metadata, caching — using the same patterns.

18.6 Where to go next

You now understand FastAPI deeply enough to build real systems. Productive next steps: add an `Order` resource with relationships to `Book` and `User`; introduce Alembic migrations; add Redis caching for hot reads; build a background worker with Celery for emails; and add structured logging and metrics. Each extends the architecture you already have.

18.7 Final recap

You began with a single `{"message": "Hello"}` endpoint and finished with a layered, authenticated, tested, containerized API backed by a real database. The throughline was always the same: declare your data and intent with type hints and models, and let FastAPI handle validation, serialization, documentation, and injection. That principle scales from your first endpoint to a production system serving millions of requests.

Appendix A — HTTP Status Codes Reference

2xx Success - 200 OK — standard success for GET, PATCH, PUT. - 201 Created — resource created (POST). - 204 No Content — success with empty body (DELETE).

4xx Client Errors - 400 Bad Request — malformed or semantically invalid request. - 401 Unauthorized — authentication missing or invalid. - 403 Forbidden — authenticated but not permitted. - 404 Not Found — resource does not exist. - 409 Conflict — conflicts with current state (duplicates). - 422 Unprocessable Entity — validation failed (FastAPI default). - 429 Too Many Requests — rate limit exceeded.

5xx Server Errors - 500 Internal Server Error — unhandled exception. - 502 Bad Gateway / 503 Service Unavailable — upstream or capacity issues.

Appendix B — Common Pitfalls and How to Avoid Them

- **Blocking calls inside `async def`.** Calling a synchronous, slow library inside an async endpoint blocks the whole event loop. Use a plain `def` endpoint (FastAPI threads it) or an async library.
 - **Returning the table model directly without a `response_model`.** You may leak fields like `hashed_password`. Always define and use read models.
 - **Forgetting `session.refresh()` after create.** Without it, the DB-generated `id` is not populated on your object.
 - **Using `exclude_unset` incorrectly on updates.** For partial updates, you *want* `exclude_unset=True` so untouched fields are preserved.
 - **Hardcoding secrets.** Keep them in environment variables via `pydantic-settings`.
 - **Mutable default arguments / shared state.** Use defaults like `Field(default_factory=list)` for mutable defaults in models.
 - **No pagination.** Always cap list endpoints with `limit`.
 - **Committing the `.env` file.** Add it to `.gitignore`.
 - **One model for everything.** Separate create/read/update schemas prevent over-posting and information leaks.
-

Appendix C — Further Reading

- **Official FastAPI documentation** — the tutorial and advanced user guide are excellent and kept current.
 - **SQLModel documentation** — covers relationships, async, and migrations in depth.
 - **Pydantic v2 documentation** — validators, settings, and serialization.
 - **Starlette documentation** — the underlying web toolkit (middleware, WebSockets, responses).
 - **OWASP API Security Top 10** — essential reading before exposing any API publicly.
 - **Alembic documentation** — production database migrations.
-

End of book. You built the Bookstore API from a single endpoint to a complete, deployable system. Keep building.